

LOONGX: Rethinking Random Graphs for AI Training Clusters

Heng Xu[♦], Yiming Lei[♡], Jialong Sun[♦], Ying Zhou[♦], Yiting Xia[♡], Jialong Li[♦]

[♦]Shenzhen University of Advanced Technology, [♡]Max Planck Institute for Informatics

ABSTRACT

Multi-tenant AI training workloads are dominated by communication within each job’s assigned nodes, while still requiring bounded network diameter as larger jobs span broader regions of the cluster. Existing network topologies struggle to simultaneously provide dense local communication and low diameter under this workload structure. We present LOONGX, a locality-biased small-world topology for multi-tenant AI training clusters. LOONGX concentrates connectivity within structured local neighborhoods to support dense intra-job communication, while allocating only a small fraction of links to random long-range connections. By leveraging the small-world effect, LOONGX achieves expander-like diameter while preserving dense local connectivity. Large-scale simulations with five representative AI training models show that LOONGX reduces iteration time by up to 39% in isolated settings and up to 58% under multi-tenant training compared to widely used baselines. A testbed implementation further confirms that LOONGX’s performance trends are consistent with simulation results.

1 INTRODUCTION

AI training at scale places substantial pressure on the network [11, 16, 25, 31, 44]. Modern training jobs involve frequent, tightly synchronized communication, making performance highly sensitive to delays along shared network paths. In this paper, we focus on multi-tenant AI clusters, where many concurrent training jobs with heterogeneous sizes and communication demands coexist and evolve over time. As a result, synchronized traffic from multiple jobs increasingly overlaps on shared network resources, amplifying the impact of path length and structural bottlenecks. This creates a fundamental tension: *training traffic is largely confined within job regions, yet the network must still provide short global paths when jobs scale or fragment.*

Clos fabrics remain the default in cloud datacenters [3] and many production AI clusters [16, 34, 43]. Their multi-stage hierarchy offers high bisection bandwidth and operational simplicity, but synchronized traffic often traverses multiple aggregation layers, creating shared bottlenecks under concurrent communication.

Expander-inspired topologies [50, 52, 59], constructed via uniformly random connectivity¹, achieve near-optimal diameter and short average path length, making them attractive from a graph-theoretic perspective. However, such uniform

randomness is poorly aligned with AI training workloads. The limitation of expander-style designs is not their diameter, but the uniform allocation of connectivity that ignores the strong spatial skew in training workloads. Training jobs are typically placed on nearby nodes, with most traffic confined within a localized region, yet expanders distribute links evenly across the entire cluster. As a result, local communication often traverses multi-hop paths spanning distant regions, increasing synchronization latency and contention. This weak locality renders expander topologies inefficient for AI training despite their favorable global properties.

Other structured topologies exhibit different limitations. Torus-based designs preserve local adjacency [25], but their diameter grows quickly with scale, pushing medium and large training jobs onto long paths. Dragonfly reduces diameter by organizing nodes into groups with dense internal connectivity [26, 49], but once jobs span multiple groups, synchronization performance degrades due to reliance on cross-group communication and becomes highly sensitive to placement. More recent structured designs, such as ZCube [60], introduce additional structural layers, incurring higher cost and complexity, and are primarily designed for single large job training rather than multi-tenant settings. These constraints limit the effectiveness of existing designs.

Taken together, these limitations reveal a common set of structural requirements for multi-tenant AI clusters. Communication is largely confined within the nodes assigned to each job, calling for strong local connectivity. At the same time, larger jobs and fragmented placements inevitably span broader portions of the cluster, making bounded network diameter necessary to prevent excessive hop counts. While Torus-based designs provide dense local connectivity, their diameter grows quickly with scale. Furthermore, the network structure should provide consistent communication properties across different regions, avoiding the placement sensitivity observed in group-based designs like Dragonfly, where performance degrades once jobs cross structural boundaries. An effective topology must therefore satisfy the combined requirements of local connectivity, bounded diameter, and structural consistency to support AI training.

To meet the above requirements, we design LOONGX, a topology that *preserves the near-optimal diameter of random graphs while introducing dense local neighborhoods* aligned with AI communication structures. It allocates most connectivity among nearby nodes so that the majority of traffic follows short local paths, and augments this structure with

¹We use Expander graph and random graph interchangeably in this paper.

a small number of randomized long-range links. This design exploits the small-world effect, where adding only a few random shortcuts to an otherwise structured network rapidly collapses the diameter. Unlike expander graphs that distribute connectivity uniformly across the network, LOONGX limits randomness to a small fraction of links, retaining around 20% randomized long-range connections that suffice to match the diameter and average path length of fully random expanders. In addition, LOONGX maintains a regular and uniform construction across the cluster, ensuring consistent communication properties and avoiding the structural sensitivity observed in group-based designs.

We evaluate LOONGX using five representative large AI training models under both isolated and multi-tenant settings. In isolated training, LOONGX reduces iteration time by up to 39% compared to Dragonfly, and by 16–29% compared to Clos-based designs. Under multi-tenant training, LOONGX reduces iteration time by up to 58% compared to Dragonfly, 21–26% compared to Clos-based designs, and up to 13% compared to Expander. We further validate LOONGX through a testbed implementation, observing performance trends consistent with simulation.

This paper makes the following contributions.

- We characterize the mismatch between AI training communication patterns and existing network topologies, and identify the structural limitations of widely used designs, including Clos, Torus, Dragonfly, Expander, and ZCube, in multi-tenant environments.
- We introduce LOONGX, a new topology that combines dense local connectivity with a small fraction of random long-range links. By leveraging the small-world effect, LOONGX achieves near-optimal diameter while preserving locality aligned with multi-tenant AI communication patterns.
- Through extensive large-scale simulations on representative AI training workloads, we show that LOONGX reduces iteration time under both isolated and multi-tenant settings, and further validate these results through a testbed implementation.

[This work does not raise any ethical issues.]

2 BACKGROUND AND MOTIVATION

In this section, we describe AI workload characteristics (§2.1), the resulting topology requirements for AI training (§2.2), the limitations of existing topologies (§2.3), and the key observation that motivates LOONGX (§2.4).

2.1 Characteristics of AI Workloads

Multi-tenant variability. Multi-tenant AI clusters routinely serve many concurrent training jobs that differ in scale, duration, and communication demand. Small and medium jobs issue modest but frequent exchanges, while larger jobs involve more workers and generate substantial cross-rack

traffic. The resulting aggregate load varies over time as jobs arrive, complete, or resize. Prior studies of Meta [16] and Alibaba [57] production traces reveal a wide spread of training job sizes, where many modest-scale jobs coexist with a smaller number of large ones. A topology must therefore support heterogeneous and time-varying demand rather than target a single workload profile.

Parallelism heterogeneity. Within each job, different parallelization strategies induce distinct communication patterns that stress the network in different ways. Data parallelism (DP) [28, 46] relies on synchronized collective operations across workers. Pipeline parallelism (PP) [24, 38] generates sequential stage-to-stage transfers that favor short paths with low latency variance. Tensor parallelism (TP) [48] partitions layers across workers and produces fine-grained exchanges that are sensitive to load imbalance. Expert parallelism (EP) [14, 32, 45, 61] create concentrated all-to-all traffic with skewed but iteration-consistent destinations. While these patterns differ in structure and granularity, they recur across iterations and collectively exercise path length, spatial locality, and load balance in the underlying topology.

Synchronization sensitivity. Many large-scale training jobs progress in synchronized iterations, where workers exchange collective communication and must complete communication before computation resumes. Under such execution models, job progress is directly affected by hop count along communication paths. As a result, short paths and sufficient local connectivity are particularly important for reducing iteration time, especially for latency-critical or fine-grained exchanges.

2.2 Topology Requirements for AI Training

AI training workloads impose specific structural requirements on network topologies, which we summarize as the key properties most directly affecting training performance.

Path length. AI training progresses in synchronized iterations, where communication latency directly accumulates into iteration time. Short path lengths reduce synchronization delay and limit the amplification of queueing effects during collective operations.

Local connectivity. AI training traffic is dominated by communication among workers of the same job, which are typically placed close to each other within the cluster. Strong local connectivity supports dense collectives and fine-grained exchanges while minimizing interference with unrelated traffic on distant links.

Long-range connectivity. While communication is often localized, large jobs and certain parallelization strategies still require efficient coordination across distant workers. Adequate long-range connectivity provides shortcuts that limit diameter growth and prevent global communication phases from becoming bottlenecks.

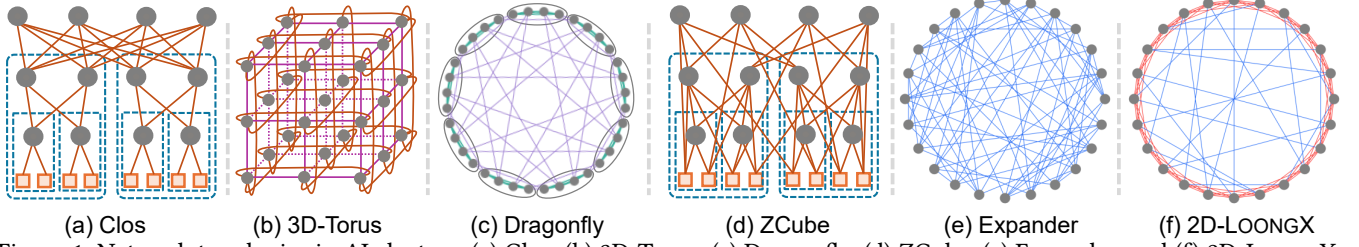


Figure 1: Network topologies in AI clusters: (a) Clos, (b) 3D-Torus, (c) Dragonfly, (d) ZCube, (e) Expander, and (f) 2D-LOONGX.

Table 1: Comparison of representative topology families.

Topology	Path length	Local connectivity	Long-range connectivity	Structural consistency	Cost
Clos	★★	★★	★★	★★	★★
Torus	★	★★★	★	★★★	★★
Dragonfly	★★★	★★★	★★	★	★★
ZCube	★★★	★★★	★★	★★	★
Expander	★★★	★	★★★	★★★	★★
LoongX	★★★	★★★	★★	★★★	★★

Structural consistency. Multi-tenant AI clusters place jobs dynamically across the network, and training performance should not depend sensitively on where workers are assigned. Structural consistency ensures that nodes experience similar path lengths and connectivity characteristics, avoiding systematically worse communication conditions.

Cost. AI training clusters must scale under practical hardware and wiring constraints, making cost an unavoidable consideration in topology design.

2.3 Why Existing Topologies Fall Short for AI Workloads

We examine why existing data center topologies fall short for AI training workloads. As illustrated in Fig. 1, current AI clusters mainly adopt two classes of designs: hierarchical topologies, such as Clos and ZCube, and flat topologies, including Torus, Dragonfly, and Expander. Table 1 compares these representative topology families along key properties. The comparison shows that most designs optimize one or two properties at the expense of others. Consequently, existing topologies cannot simultaneously provide short paths, strong locality, and structural consistency for multi-tenant AI training under practical cost constraints.

Clos. Clos designs [3] are widely deployed in production data centers due to their symmetric bisection bandwidth and simple wiring; they perform well for cloud workloads dominated by independent, throughput-driven flows. However, AI training relies on synchronized, iteration-bound collectives such as all-reduce and all-gather. In large Clos fabrics, these collectives traverse multiple stages over long multi-hop paths, increasing latency and exposing communication to repeated contention along the path. As a result, iteration time becomes highly sensitive to path length and contention.

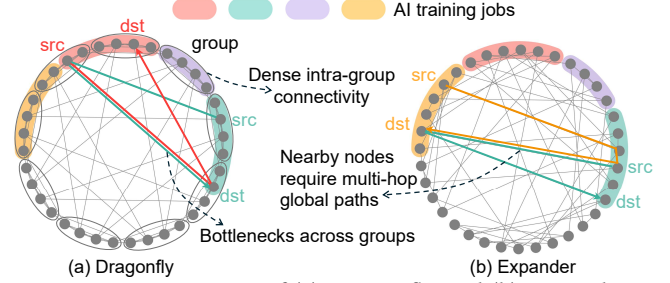


Figure 2: Limitations of (a) Dragonfly and (b) Expander.

Recent Rail-optimized variants [1, 54] further tailor Clos fabrics to specific parallelization patterns, but they preserve the same hierarchical structure. Their benefits rely on well-aligned job placement and communication patterns, and they diminish for all-to-all communication or multi-tenant training jobs.

Torus. Torus topologies are used in accelerator-centric systems, including Google TPU deployments [25, 63], as they provide dense local connectivity. However, their diameter grows with system scale, resulting in long multi-hop paths for distant communication and the lack of efficient shortcuts. This makes Torus particularly inefficient for all-to-all communication and amplifies synchronization delays as job sizes grow or placements fragment.

Dragonfly. Dragonfly [26, 49] reduces diameter by organizing nodes into fixed groups with full-mesh internal connectivity, which supports efficient communication within a group, and connecting groups through a small number of global links. As shown in Fig. 2a, communication across groups is forced onto these limited inter-group paths once a job spans multiple groups, which quickly become bottlenecks. Consequently, communication performance becomes highly placement-sensitive, limiting structural consistency for multi-tenant AI training.

ZCube. Structured topologies such as ZCube [60] achieve low diameter through symmetric, recursive construction with relatively uniform hop counts. The connectivity pattern is tightly coupled to the node count and per-node port budget, requiring multi-port NICs and increasing hardware cost. Ports are assigned to different layers in a fixed manner, which limits flexibility in rebalancing local and long-range connectivity. ZCube is designed and evaluated primarily for single

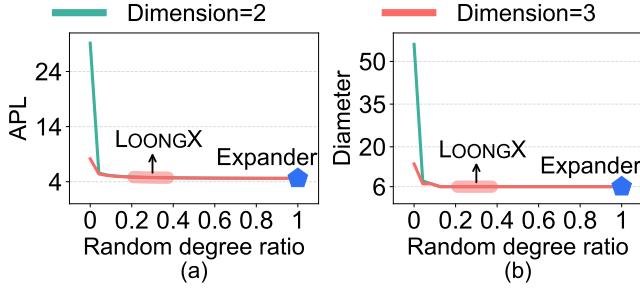


Figure 3: (a) APL and (b) diameter under varying random degree ratios at 10k-nodes.

training jobs, and its behavior under concurrent multi-tenant workloads has not been explored. Under dynamic placement, communication performance can therefore depend on how jobs align with the predefined structure.

Expander Topologies. Expander topologies [50, 52, 59], typically constructed using *random graph* techniques, rely on uniformly random connectivity to achieve small diameter. As shown in Fig. 2b, this randomness means that even physically nearby nodes may lack direct links and must communicate via multi-hop paths through distant nodes. For AI training workloads, where communication is dominated by dense exchanges among nearby workers, this weak local connectivity forces traffic onto global paths and degrades synchronization efficiency.

2.4 Key Observation Behind LOONGX

LOONGX builds on the small-world effect [7, 39, 51], where introducing a small fraction of random shortcuts into an structured network can dramatically reduce diameter and average path length (APL). As shown in Fig. 3, both APL and diameter decrease rapidly as a small amount of randomness is introduced, but quickly saturate thereafter, with the fully random expander representing the lower bound. In particular, with dimension $w = 2$, allocating only 20% of uplinks to random shortcuts increases APL by just 5.1% compared to an expander, and this gap further shrinks to 3.4% at 30% randomness. Leveraging this property, allocating only a limited fraction of edges to random shortcuts can achieve expander-like diameter and near-optimal APL, while reserving the remaining degree for structured local connectivity.

Fig. 4 illustrates the relationship between topological randomness and iteration time on a 2.5k-nodes LOONGX topology. We run five concurrent DeepSeek-671B training jobs, each occupying 512 nodes; additional simulation details are provided in §5.1. The random degree ratio denotes the fraction of uplink ports assigned to random shortcuts.

At low randomness below 0.2, the limited number of shortcuts leads to large diameter and APL, causing even modest long-distance communication to traverse excessive hops and degrade performance. As randomness increases to the range

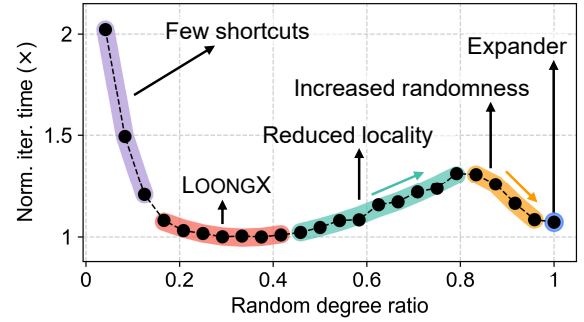


Figure 4: Design space of LOONGX.

of 0.2–0.4, the topology simultaneously achieves dense local connectivity and low network diameter, resulting in the lowest iteration time. When randomness further increases to 0.4–0.8, additional randomness provides little further reduction in diameter or APL, but increasingly disrupts structural locality and degrades short-range communication efficiency. At very high randomness above 0.8, locality is largely lost, but strong structural consistency reduces sensitivity to task placement, leading to a partial performance recovery that remains inferior to the optimal range of 0.2–0.4.

3 DESIGN

In this section, we present the design of LOONGX, including its construction (§3.1), fundamental properties (§3.2), routing strategy (§3.3), and job placement (§3.4).

3.1 LOONGX Construction

While LOONGX is not an expander graph per se, we use expander constructions as a reference point to introduce and contextualize the LOONGX topology. Expander or random regular graphs can be defined as $EX(n, d)$, where n is the number of nodes and each node has d uplinks². A standard expander construction connects each node to d uniformly chosen peers while avoiding self-loops and duplicate edges. The left part of Fig. 5 shows an expander graph with $n = 36$ and $d = 5$, in which all edges are randomly generated. We refer to these edges as global edges to distinguish them from the structured local edges introduced in LOONGX.

Random shortcuts r . We denote LOONGX as $LX(n, d, r, w)$. Compared to an expander $EX(n, d)$, LOONGX introduces two additional parameters: the number of random edges r and the number of structured dimensions w . In particular, an expander corresponds to the special case $r = d$, where all d uplinks are used for random connections. Each node in LOONGX partitions its d ports into two components:

- r random edges providing global connectivity,
- $d - r$ structured local edges.

The random degree r controls global connectivity. As discussed in §2.4, we set r/d to 0.2–0.4, which is sufficient to

²Here, d denotes the number of uplink ports, rather than the total port count of a switch, for simplicity of exposition.

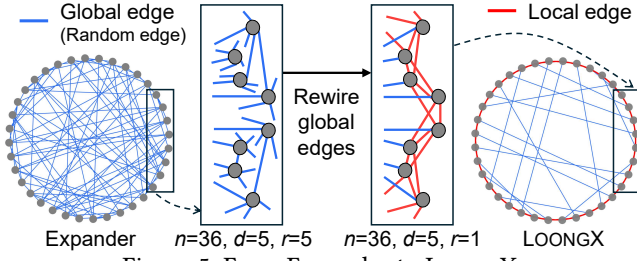


Figure 5: From Expander to LOONGX.

achieve expander-like diameter while preserving strong local connectivity. Fig. 5 shows an example of transforming an expander into LOONGX. Starting from an expander with $n = 36$ and $d = 5$ ($r = 5$), LOONGX with $w = 2$ reduces the number of random edges to $r = 1$ and reallocates the remaining $d - r = 4$ ports to structured local connections.

Structured dimensions w . The $d - r$ local edges are organized into w independent dimensions. Each dimension i is allocated d_i local ports, such that $\sum_{i=1}^w d_i = d - r$. Within each dimension, local edges connect each node to its nearest neighbors under the dimension-specific ordering, assigning at most one edge per neighbor, until the d_i local ports in that dimension are exhausted.

The choice of w and $\{d_i\}$ determines how local connectivity is organized. Specifically, $d - r$ controls the overall strength of local connectivity, while w determines the geometric structure of the local region across multiple dimensions. Increasing w allows local regions to expand along multiple independent directions, offering greater flexibility in organizing structured connectivity.

The $LX(n, d, r, w)$ parameterization defines a continuous design space between fully structured and fully random topologies:

- $r = d$, $LX(n, d, r, w)$ becomes $EX(n, d)$;
- $r = 0$, $LX(n, d, r, w)$ reduces to a purely structured topology;
- $r = 0$ and $d/w = 2$, $LX(n, d, r, w)$ corresponds to a w -dimensional Torus.

LOONGX is constructed in two steps as outlined in Alg. 1, with the detailed pseudocode for its subroutines provided in Appx. C. First, **BUILDSTRUCTUREDBACKBONE** (Alg. 2) establishes the structured backbone using the $d - r$ local ports and the chosen w -dimensional layout, forming structured local connectivity. Second, **ADDRAMASHORTCUTS** (Alg. 3) adds r random shortcut edges to each node via uniform sampling while avoiding duplicates and self-loops.

3.2 Properties of LOONGX

Low diameter. LOONGX combines a structured local backbone with a small number of random shortcuts. Our results in §2.4 show that when only 20% of uplinks are allocated to random shortcuts, the resulting topology achieves expander-like diameter, while its APL is only about 13% higher. As a

Algorithm 1 LOONGX CONSTRUCTION

Require:

- $\mathcal{R}$: set of nodes, $|\mathcal{R}| = n$;
- d : number of uplinks per node;
- r : number of random edges, $0 \leq r \leq d$;
- w : number of dimensions;
- $\{d_i\}_{i=1..w}$ with $\sum_{i=1}^w d_i = d - r$;
- $\mathcal{L}$: structured layout policy for w dimensions.

Ensure:

- ```

 $LX(n, d, r, w) = (V, E) \leftarrow$ simple undirected d -regular graph
over $V = \mathcal{R}$.
1: $V \leftarrow \mathcal{R}; \quad E \leftarrow \emptyset$ // V denotes the vertices, and E denotes the edges.
 // Step 1: build the $(d - r)$ -regular backbone (see Alg. 2 for details)
2: $E_{\text{local}} \leftarrow \text{BUILDSTRUCTUREDBACKBONE}(V, d - r, \mathcal{L})$
 // Step 2: add a Ramanujan shortcut layer (see Alg. 3 for details)
3: $E_{\text{short}} \leftarrow \text{ADDRAMASHORTCUTS}(V, r, E_{\text{local}})$
 // Merge edges; resulting degree per node is $(d - r) + r = d$
4: $E \leftarrow E_{\text{local}} \cup E_{\text{short}}$
5: return $LX(n, d, r, w) = (V, E)$

```

result, LOONGX maintains short path lengths even at large scales, enabling efficient communication across many nodes.

**Locality preservation.** Unlike purely random topologies, LOONGX incorporates structured local connectivity that gives rise to strong locality. We quantify locality using the *clustering coefficient*<sup>3</sup>, defined as the probability that two neighbors of a node are also connected to each other. As shown in §5.2 and Table 4, LOONGX achieves a substantially higher clustering coefficient than Expander and Torus, and is comparable to that of Dragonfly, which exhibits strong locality through fully connected groups.

For fixed  $n$  and  $d$ , increasing the dimension  $w$  tends to reduce the clustering coefficient, as nodes become more sparsely connected in higher-dimensional structures. We formalize this relationship in Theorem 3.1.

**Structural consistency.** The LOONGX structure exhibits a uniform local geometry across the topology: each node connects to nearby neighbors through  $d - r$  structured ports along  $w$  dimensions, and different parts of the topology share similar local connectivity patterns. As illustrated by the zoomed-in view of LOONGX in Fig. 5, every node has the same local connectivity structure. This uniformity allows jobs of different sizes to be placed on parts of the topology with similar geometric properties. Intuitively, LOONGX can be viewed as many similar local units, lightly connected by a small number of global shortcuts.

**Path diversity.** Structured local connectivity provides deterministic short paths that preserve locality for training traffic. Adding a fraction  $r$  of random shortcuts introduces long-range alternatives and increases path diversity, allowing traffic to spread across multiple routes under load and reducing concentration on individual bottlenecks.

<sup>3</sup>The clustering coefficient is defined for flat topologies and is not directly applicable to hierarchical designs such as Clos or ZCube.

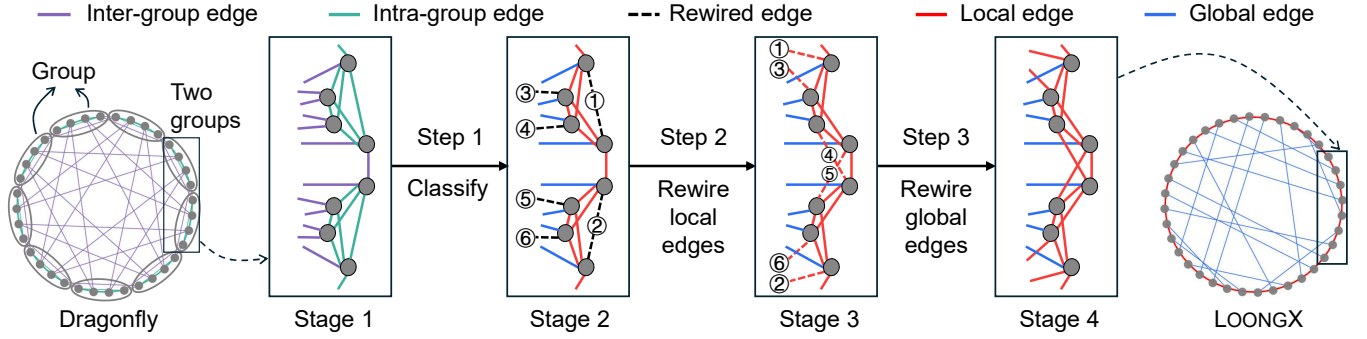


Figure 6: From Dragonfly to LOONGX.

### 3.3 Routing

We follow common practice for expander-like networks and use K-shortest path (Ksp) routing [36, 50, 52]. Ksp serves as a standard baseline to enumerate multiple available paths and does not introduce additional structure or optimization. Its role is solely to expose the path diversity inherent in the topology, rather than to create it.

Under this routing setup, LOONGX's locality-biased small-world structure naturally yields different path characteristics for local and global traffic. For communication confined to nearby nodes, the dense local connectivity provides many short paths, allowing Ksp to select low-hop routes similar to those in fully connected subgraphs. For larger jobs whose communication spans a wider portion of the cluster, the presence of random shortcut edges keeps end-to-end paths short even across distant nodes, and Ksp exposes multiple alternative routes through the structured backbone to reach different shortcuts.

### 3.4 Job Placement

Job placement operates under two common situations: when a job can be assigned a single contiguous block of nodes, and when fragmentation requires a non-contiguous allocation. While placement could in principle be optimized based on job-specific communication patterns, we intentionally adopt a simple placement mechanism without traffic-aware tuning. Our goal is to expose the advantages of LOONGX's topology itself, rather than to improve training performance through complex placement algorithms.

**Contiguous block selection.** LOONGX's structured backbone induces a stable topological ordering of nodes. Along each logical dimension, nodes follow a regular adjacency pattern, and together these dimensions define a backbone ordering that can be traversed sequentially. Placement selects a block of nodes that is contiguous along this backbone, with the block size chosen to match the job's requirement. The block can be extended incrementally along one or multiple dimensions, allowing the same procedure to support jobs of different scales without predefined region boundaries.

**Non-contiguous block selection.** In multi-tenant clusters, a fully contiguous block may be unavailable due to occupied nodes. In this case, placement follows the same backbone ordering and selects nodes from blocks that are adjacent along this ordering. The allocator assembles a limited number of such blocks until the job's requirement is met, providing a simple and consistent fallback under fragmentation.

## 4 DEPLOYMENT CONSIDERATIONS

Here we discuss deployment considerations for LOONGX, focusing on cabling and wiring (§4.1), incremental expansion (§4.2), cost analysis (§4.3), and failure handling (§4.4).

### 4.1 Cabling and Wiring

While LOONGX introduces a new connectivity structure, its cabling can be derived from existing topologies rather than built from scratch. In particular, LOONGX can be obtained by rewiring existing Expander and Dragonfly topologies.

**Rewiring from an Expander.** Starting from a random regular expander with degree  $d$ , we rewire  $d - r$  edges per node to form structured local connections among physically nearby nodes, while retaining only  $r$  random edges for long-range connectivity (Fig. 5). The rewired edges follow fixed and repeatable local wiring patterns, creating dense local connectivity while keeping most links short and spatially constrained. Compared to a fully random expander, this construction reduces cabling complexity by limiting the number of long cables, making the topology more amenable to practical deployment in large-scale AI clusters.

**Rewiring from a Dragonfly.** LOONGX can also be obtained by rewiring an existing Dragonfly topology. As illustrated in Fig. 6, we first classify Dragonfly links into dense intra-group edges and sparse inter-group edges. Links that already conform to the target LOONGX structure, including both local and long-range connections, are preserved, while the remaining links are designated as rewiring candidates. These candidate links are then reassigned to complete the structured local connectivity required by LOONGX. Any leftover links are finally repurposed as global edges to provide long-range connectivity. This workflow prioritizes the formation

of LOONGX's structured local connectivity while maximizing the reuse of existing links.

## 4.2 Incremental Expansion

A key challenge in large-scale AI clusters is enabling incremental expansion without disruptive rewiring. LOONGX addresses this challenge by decoupling local structural growth from global random connectivity: new nodes are integrated through structured local dimensions, while only a small number of random shortcuts are adjusted to preserve global properties. The detailed procedure of incremental expansion is described in Alg. 4 in Appx. E.

**Structured local expansion.** Expander topologies are difficult to grow incrementally because each node uses all  $d$  ports for global connectivity, so adding a new node often requires reconfiguring many existing random links. In contrast, LOONGX explicitly encodes locality through structured dimensions that define stable local neighborhoods. This structure allows new nodes to be integrated by extending these dimensions with only localized rewiring, while leaving the rest of the topology unchanged.

**Incremental global shortcut assignment.** To preserve global connectivity during expansion, each node in LOONGX maintains only  $r$  random shortcuts. When adding a new node, incorporating it into the global structure requires modifying only a small number of long-range edges, rather than reconfiguring all  $d$  connections. Assigning  $r$  new shortcuts is sufficient to maintain low diameter and short path length.

## 4.3 Cost Analysis

From a cost perspective, LOONGX inherits the advantages of Expander. LOONGX uses the same number of switches and per-node degree as an Expander, differing only in how a fraction of links are rewired to form structured local connectivity. As a result, the total number of switches, inter-switch links, and overall cabling volume remain comparable to those of an Expander, allowing LOONGX to retain its well-established cost benefits, namely requiring only about 80–85% of the switching hardware of Clos under the same host scale [52].

The detailed price breakdown is shown in Table 6, with pricing obtained from prior work [31, 55]. We primarily adopt the MixNet [31] cost model, and additionally reference raw ATOP [60] cost data where available. When applied to the same topologies, the two models differ by only about 10–20% in absolute cost estimates due to pricing assumptions, indicating that our cost estimation closely reflects the underlying cost trends reported in prior work. Across all evaluated scales (Table 4), LOONGX incurs comparable cost to Expander, consistent with their identical switch counts and cabling requirements, while Zcube, which introduces an additional structural layer, consistently incurs higher cost. Overall, these results show that LOONGX introduces no additional baseline cost.

## 4.4 Failure Handling

**Local failover via structured connectivity.** LOONGX's structured backbone provides multiple deterministic directions between nearby nodes. Under local link or node failures, Ksp selects alternative paths that avoid failed components, preserving locality with only a modest increase in hop count.

**Global failover via random shortcuts.** Each node also maintains a small number of random shortcuts connecting distant regions. These shortcuts provide independent long-range alternatives allow flows to bypass failed regions without incurring long detours.

**Path stretch under failures.** By combining structured local alternatives with random global shortcuts, LOONGX limits path stretch under common failure scenarios. As shown in Fig. 17, even under a 10% link failure rate, the average path length increases by less than 2%.

## 5 EVALUATION

In this section, we cover simulation setup (§5.1), followed by topology characteristics (§5.2), performance under non-AI traffic (§5.3), isolated AI workload performance (§5.4), multi-tenant AI workload performance (§5.5), and evaluation under various settings (§5.6).

### 5.1 Simulation Setup

**Simulation framework.** We use htsim [9], a discrete-event, packet-level simulator widely adopted for evaluating data center networks [20, 27, 36]. Its packet-level granularity enables high-fidelity modeling of network dynamics. However, the native simulator only supports non-AI traffic, where flows are independent of one another, and does not capture the flow dependencies in AI training workloads. To address this limitation, we extend htsim with a directed acyclic graph (DAG) scheduler that models task-dependency graphs in AI training. More details are provided in Fig. 19 in Appx. B.

**LOONGX parameter setting.** Recall that LOONGX is modeled as  $LX(n, d, r, w)$ . We evaluate networks with  $n = 128, 320, 640$ , and 1280 switches, corresponding to approximately 1k, 2.5k, 5k, and 10k nodes, respectively. Unless otherwise specified, we use a default configuration of  $d = 24$  and  $r = 6$ , corresponding to 25% random shortcuts, and set the dimension to  $w = 2$ . In §5.6, we further explore the design space by varying  $d$ ,  $r$ , and  $w$ . More details on the network settings of LOONGX and baselines are provided in Table 4.

**Topology baselines.** We compare LOONGX with two representative classes of network architectures. The first class consists of hierarchical topologies, namely the industry-standard Clos [3] and ZCube [60], the latter designed for large model training. The second class consists of flat topologies, namely Torus [25], Dragonfly [26] commonly used in HPC systems, and Expander [21] derived from random-graph theory. We evaluate networks with 1k, 2.5k, 5k, and 10k nodes, with configurations summarized in Table 4.

**Workloads.** We consider two traffic classes: generic data center traffic (non-AI) and AI training workloads. For non-AI traffic, we use the Websearch [4] and Meta traces [16], with flow-size distributions shown in Fig. 21, and set the core network load to 60% to represent a heavy-load regime. In production data centers, median link utilization is typically 10%–20% [47], with over 80% of links operating below 10% utilization [62]. For AI training workloads, we evaluate five representative large models: DeepSeek-671B [32], Qwen3-MoE [61], GLaM-64B/64E [14], GPT-MoE [45], and GPT-175B [10]. These workloads are generated using the AICB library in SimAI [56]. Operator-level computation is profiled on a 16-nodes GPU cluster with four H200 GPUs per node and 120 GB memory each, and communication traces are derived from model specifications and parallelization strategies and scaled to different network sizes. Detailed configurations are shown in Table 2 in Appx. A. In addition, we include a dense all-to-all workload, where each node sends 1 GB to every other node.

**Routing policy.** We adopt routing strategies based on topology type. For hierarchical topologies, namely Clos and ZCube, we enable Equal-Cost Multi-Path (ECMP) [23] to distribute traffic across available paths. For flat topologies, namely LOONGX, Torus, Dragonfly, and Expander, we use Ksp [50, 52] with the 8 shortest paths to exploit path diversity and balance load. By default, each flow uses a single path. We also evaluate finer-grained load balancing using packet-level random spray and round-robin (RR) spray [35].

**Transport protocols.** We use NDP [20] as the default transport protocol and compare it with Bolt [6], HPCC [29], and ExpressPass [13]. Unless otherwise specified, all links operate at 100 Gbps with an MTU of 1.5 KB. Table 3 summarizes the detailed settings.

## 5.2 Topology Characteristics

We evaluate the topological properties of LOONGX to provide context for the subsequent results. We consider networks at a scale of approximately 2.5k-nodes and report average path length (APL), diameter, and clustering coefficient.

**APL and diameter.** Fig. 7a reports the APL and diameter for networks with  $d = 24$  uplinks. The random degree ratio  $r/d$  captures the fraction of uplink ports assigned to random shortcuts;  $r/d = 1$  yields an Expander, while  $r/d = 0$  corresponds to purely local connectivity. Both metrics improve rapidly with the introduction of a small fraction of random shortcuts. Under  $w = 2$ , the Expander achieves an APL of 4.07, while LOONGX with  $r/d = 0.2$  increases APL only to 4.34 (6.6%). With 30% random shortcuts, the APL overhead further drops to 4.4%, beyond which additional shortcuts yield diminishing returns. A similar trend is observed for diameter: LOONGX matches the Expander diameter of 5 with only 20%

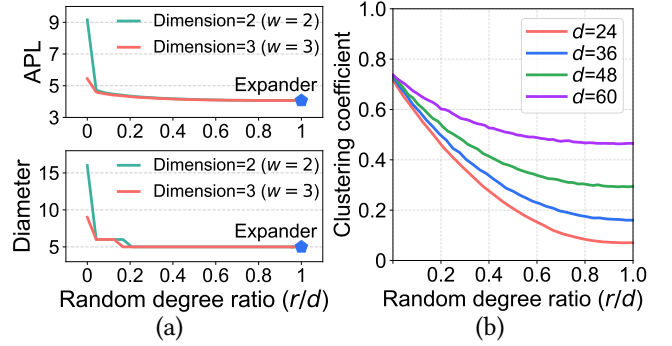


Figure 7: (a) APL, diameter, and (b) Clustering coefficient under varying random degree ratios in 2.5k nodes.

random shortcuts. Overall, LOONGX achieves Expander-like topology properties with a small random budget.

**Clustering coefficient.** Fig. 7b reports the clustering coefficient under four uplink configurations ( $d \in 24, 36, 48, 60$ ) with  $w = 2$ . As  $d$  increases, the clustering coefficient rises, reflecting stronger local connectivity enabled by higher-radix switches. In contrast, the clustering coefficient decreases monotonically with the random degree ratio, indicating that random shortcuts progressively disrupt local triangles and weaken structured locality. For example, with  $d = 24$ , an Expander exhibits a clustering coefficient of only 0.07, whereas LOONGX achieves 0.45 with  $r/d = 0.2$ , a 6.4 $\times$  improvement. Additional results comparing LOONGX with flat topologies are summarized in Table 4, further confirming its advantage in preserving dense local structure.

## 5.3 Non-AI Traffic Performance

We first evaluate non-AI workloads at the 2.5k-node scale using the Websearch and Meta trace workloads.

**FCT.** Fig. 8a and Fig. 8c compare the 99%-ile FCT across two traces. Expander achieves the lowest tail latency, followed by LOONGX, while 6D-Torus performs the worst. Under the Websearch trace, the 99%-ile FCT of LOONGX is only 18% higher than Expander, whereas Torus is approximately 5 $\times$  worse. This trend follows their diameter and APL: Expander minimizes both, Torus has the largest diameter, and LOONGX incurs a slightly higher APL. Despite this gap, LOONGX remains competitive for non-AI traffic due to its lower cabling complexity, and we will show that it outperforms Expander for AI workloads with more localized communication.

**Per-flow throughput.** Fig. 8b and Fig. 8d report the corresponding per-flow throughput [40]. Per-flow throughput is computed as the flow size divided by its FCT, which reflects the effective bandwidth obtained by each flow. Expander achieves the highest per-flow throughput, followed by LOONGX, ZCube, and Dragonfly, with 6D-Torus performing the worst. This ordering is consistent with the FCT results. Under the Meta trace, Expander achieves 25.2% higher per-flow throughput than LOONGX, 26.4% higher than ZCube,



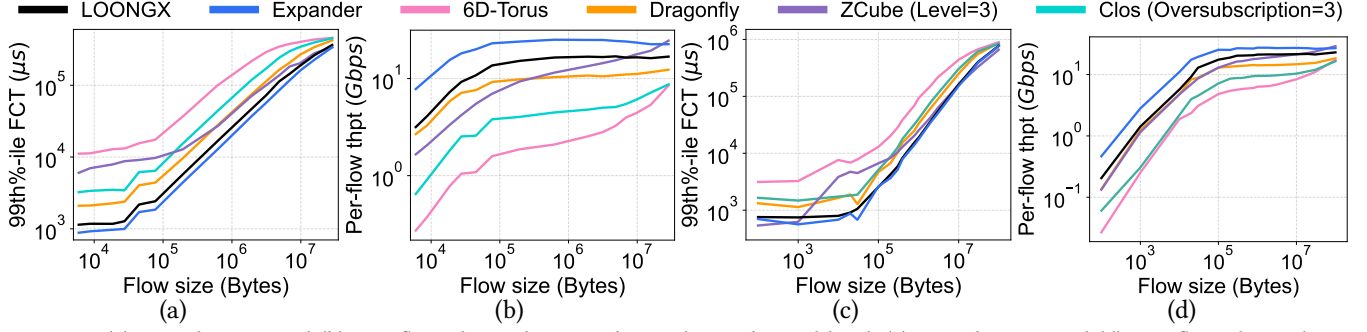


Figure 8: (a) 99%-ile FCT and (b) Per-flow throughput under Websearch workload. (c) 99%-ile FCT and (d) Per-flow throughput under Meta trace.

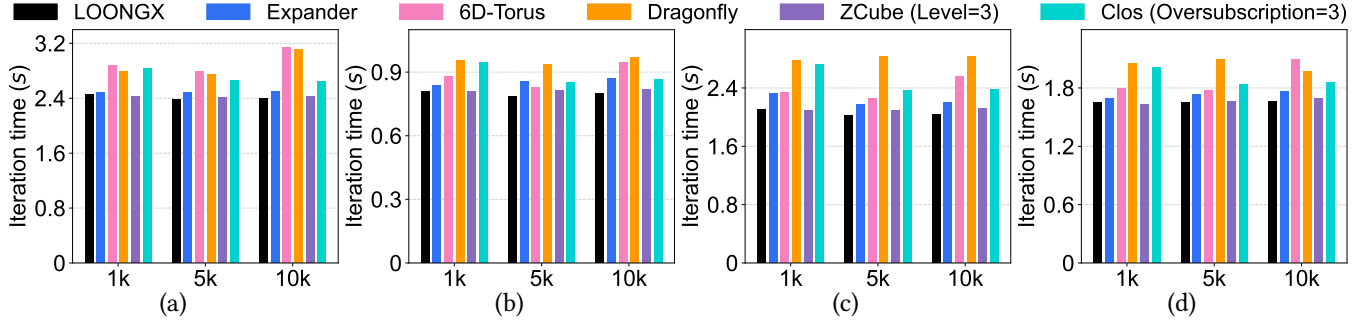


Figure 9: Iteration time across different AI cluster scales under isolated training workloads: (a) DeepSeek-671B, (b) GLaM-64B/64E, (c) GPT-MoE, and (d) Qwen3-MoE.

and 78.9% higher than Dragonfly. Overall, the per-flow throughput trends closely follow the FCT performance and can be explained by the same underlying topological properties.

#### 5.4 Isolated AI Workload Performance

We evaluate the performance of LOONGX under isolated single-job training workloads. Fig. 9 reports the iteration time across cluster sizes of 1k, 5k, and 10k nodes. We also evaluate a dense model, GPT-175B, in Appx. F, which exhibits higher tensor parallel traffic than the models discussed here.

**Compared with flat topologies.** We first compare LOONGX with flat topologies, i.e., Expander, Torus, and Dragonfly. Across all evaluated workloads and scales, LOONGX consistently outperforms these baselines. Dragonfly performs the worst, incurring up to a 39.3% higher iteration time than LOONGX under the GPT-MoE workload at the 5k-nodes scale, due to its reliance on sparse inter-group links that quickly saturate under global communication. Torus exhibits a 10%–30% slowdown, primarily caused by its larger network diameter. Expander shows a smaller but consistent gap, with iteration time increasing by 10.2% at 1k nodes and 7.6% at 10k nodes for GPT-MoE, reflecting its lack of explicit locality. By combining dense local connectivity with efficient global shortcuts, LOONGX better matches the communication patterns of large-scale AI training.

**Compared with hierarchical topologies.** We further compare LOONGX with hierarchical designs, ZCube and Clos.

ZCube achieves performance close to LOONGX by leveraging multiple NICs to expose rich path diversity and mitigate local congestion, but this comes at a higher cost and complexity, requiring approximately 1.5–2.5× the cost of LOONGX due to additional NICs and links. In contrast, Clos suffers from congestion because of limited bandwidth at upper tiers, incurring 15.8%, 16.6%, 28.8%, and 22.2% higher iteration times than LOONGX at the 1k-nodes scale across the four evaluated models. Overall, LOONGX achieves performance comparable to multi-NIC-based designs while avoiding the bandwidth bottlenecks of Clos.

#### 5.5 Multi-Tenant AI Workload Performance

**Iteration time and networking cost tradeoff.** We evaluate the trade-off between iteration time performance and networking cost under multi-tenant training, where four heterogeneous models execute concurrently and share network resources. Component cost assumptions follow prior work [31, 55] (Table 6, assuming 100 Gbps setting), and network configurations are detailed in Table 4.

**Better performance than flat topologies.** Fig. 10 shows normalized iteration time versus normalized networking cost for a 2.5k-nodes cluster. Flat topologies, including Expander, Dragonfly, and Torus, have comparable network cost to LOONGX since all are flat designs with similar switch and link counts. However, Expander incurs up to 13% higher iteration time (GPT-MoE), Dragonfly performs significantly

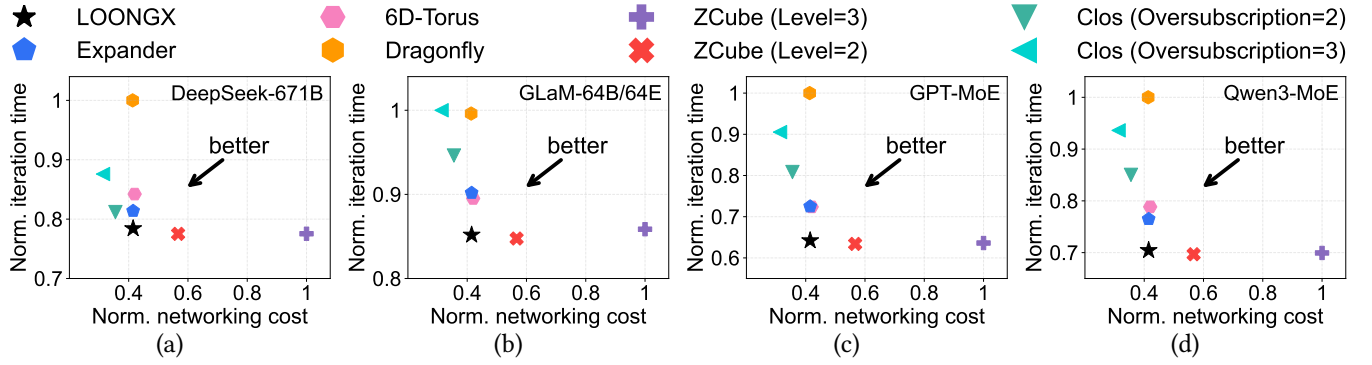


Figure 10: Iteration time vs. networking cost for concurrent workloads in a 2.5k-nodes multi-tenant AI cluster: (a) DeepSeek-671B, (b) GLaM-64B/64E, (c) GPT-MoE, and (d) Qwen3-MoE.

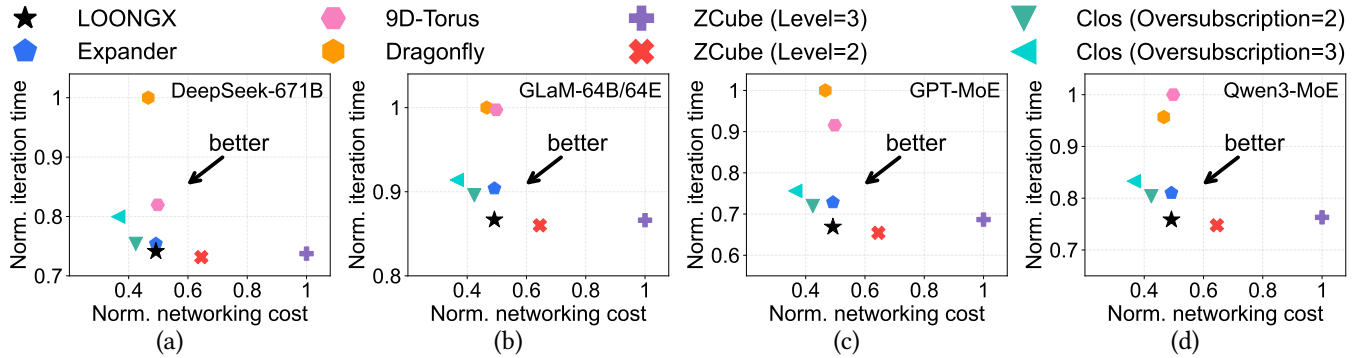


Figure 11: Iteration time vs. networking cost for concurrent workloads in a 10k-nodes multi-tenant AI cluster: (a) DeepSeek-671B, (b) GLaM-64B/64E, (c) GPT-MoE, and (d) Qwen3-MoE.

worse with a 58% increase, and Torus shows a 13% slowdown for Qwen3-MoE. These gaps arise from structural limitations: Expander lacks locality and forces local traffic to detour through distant nodes, Torus suffers from large hop counts across blocks, and Dragonfly is constrained by sparse inter-group bandwidth. By contrast, LOONGX combines dense local connectivity with a small fraction of random shortcuts, achieving both locality and low diameter, and consistently outperforming these flat topologies in multi-tenant training.

**Lower cost than ZCube.** ZCube achieves performance comparable to LOONGX by relying on multiple NICs and complex multi-plane wiring to sustain high throughput, which incurs a substantial cost premium. Specifically, ZCube (level = 3) attains only 1.1% lower iteration time than LOONGX for the DeepSeek-671B model, while requiring 2.4× higher network cost. In contrast, LOONGX achieves similar performance through architectural design rather than hardware over-provisioning: it confines dense communication to a structured local backbone while maintaining sufficient global connectivity. Therefore, LOONGX matches ZCube-level performance at lower cost, yielding a more favorable performance–cost trade-off.

**Better performance than Clos.** Clos architectures exhibit a fundamental cost–performance trade-off: oversubscription

lowers cost but induces severe congestion. With oversubscription = 2, Clos reduces network cost by 14%, yet increases iteration time by 26% for GPT-MoE and 21% for Qwen3-MoE. LOONGX sustains high performance without incurring the congestion penalties of oversubscribed Clos, achieving a more favorable trade-off. Fig. 11 presents results for a 10k-nodes cluster, where LOONGX maintains performance comparable to that in Fig. 10, indicating that its topological properties scales effectively with cluster size.

## 5.6 Performance under Various Settings

We next evaluate different design aspects of LOONGX under diverse settings. Unless otherwise specified, all experiments use a 2.5k-nodes LOONGX topology with AI workloads.

**Varying number of random edges  $r$ .** Fig. 12 shows the impact of varying the number of random edges  $r$  with a fixed uplink degree  $d = 24$ . As  $r$  increases from zero, the iteration time decreases, whereas too few random edges lead to a large network diameter (Fig. 7a), causing even a small amount of long-distance traffic to become a bottleneck due to excessive hop counts. A random degree ratio in the range of 20%–40% yields the lowest iteration time, where LOONGX balances dense local connectivity with sufficient random shortcuts to maintain a low diameter for global communication. When the random degree ratio exceeds 40%, performance degrades

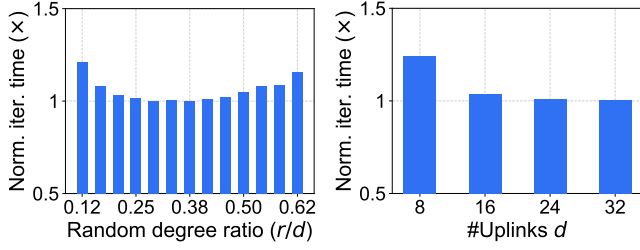


Figure 12: Impact of the number of random edges  $r$ .

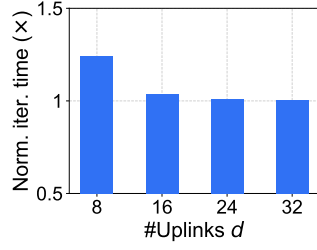


Figure 13: Impact of the number of uplinks  $d$ .

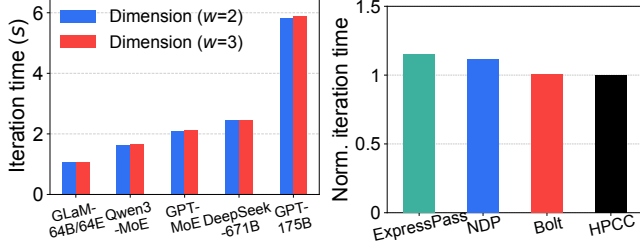


Figure 14: Iteration time under varying dimension  $w$ .

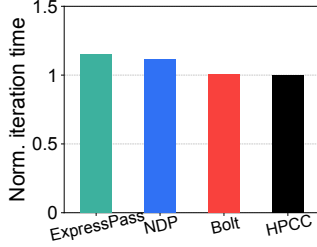


Figure 15: FCTs under different transport protocols.

as excessive randomization weakens structural locality and inflates the path length of local communication, as illustrated in the right part of Fig. 2.

**Varying number of uplinks  $d$ .** We fix the random degree ratio at 25% and vary the number of uplinks  $d$ , with the number of random edges scaling accordingly. As shown in Fig. 13, increasing  $d$  reduces iteration time, yielding a 17% improvement when  $d$  increases from 8 to 16 due to higher path diversity and fan-out. Further increasing  $d$  from 24 to 32 provides diminishing gains, indicating saturation once sufficient connectivity is reached.

**Varying dimension  $w$ .** Fig. 14 compares five AI workloads under dimensions  $w = 2$  and  $w = 3$ . Across all workloads, 3D-LOONGX achieves performance comparable to the 2D baseline, with iteration time increases of 0.53% for GLaM-64B/64E, 1.43% for Qwen3-MoE, 1.22% for GPT-MoE, 0.67% for DeepSeek-671B, and 0.64% for GPT-175B. A breakdown of computation and communication suggests that the additional network overhead from the higher-dimensional topology is small, implying minimal performance impact when extending LOONGX to higher dimensions.

**Impact of transport protocols.** Fig. 15 evaluates the interaction between transport protocols and the LOONGX topology. Bolt and HPCC achieve the best performance, while ExpressPass and NDP incur 15% and 11% higher iteration times, respectively. This behavior reflects protocol design assumptions: HPCC and Bolt rely on fine-grained, feedback-driven congestion control and adapt well to flat topologies adopting Ksp, whereas NDP and ExpressPass are primarily tailored to Clos-style hierarchical networks and are less aligned with flat connectivity.

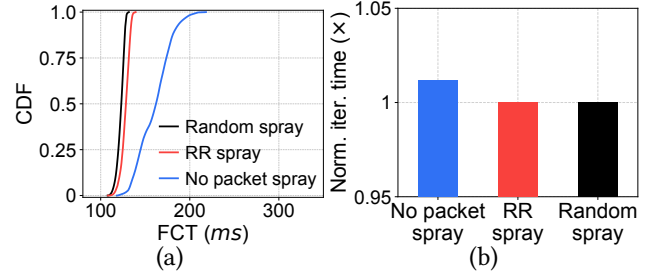


Figure 16: FCTs under different load balancing granularities for (a) all-to-all traffic and (b) the DeepSeek-671B workload.

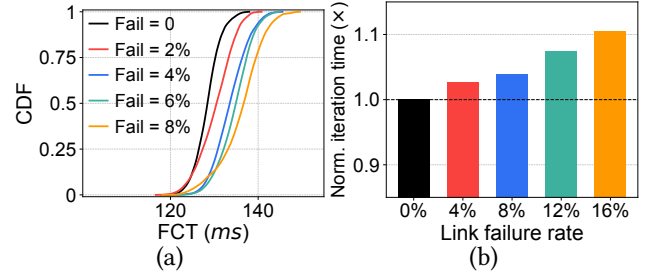


Figure 17: (a) FCT distributions for the all-to-all workload and (b) Normalized iteration time for the DeepSeek-671B workload under varying link failure rates.

**Impact of load balancing granularity.** We evaluate three load balancing granularities on two traces: dense all-to-all traffic, where each node sends 1 GB to every other node (Fig. 16a), and the DeepSeek-671B workload (Fig. 16b). The default no packet spray policy performs the worst among all schemes, exhibiting 57% and 50% higher 99%-ile FCT than random and RR spray, respectively, under dense all-to-all traffic. For the DeepSeek-671B workload with sparser communication patterns, the performance gains from packet spray are reduced. Overall, packet spray improves network performance by enhancing load balancing, at the cost of potential packet reordering overhead.

**Robustness to failures.** Fig. 17 evaluates the robustness of LOONGX under varying link failure rates for dense all-to-all traffic and a DeepSeek-671B training trace, with Fig. 17a and Fig. 17b reporting the FCT distributions and iteration time, respectively. LOONGX remains stable under link failures: for all-to-all traffic, the 99%-ile FCT increases by 5.1% at a 4% failure rate and 7.5% at a 8% failure rate, while for the sparser AI workload, iteration time increases by 2.6% at a 4% failure rate and 3.8% at a 8% failure rate. This robustness arises from dense local connectivity combined with random shortcuts, which preserve short paths while enabling efficient detours around failures.

We also compute the APL under a 10% link failure rate, denoted as  $APL_{fail}$ , as shown in Table 4. LOONGX incurs less than a 2% path length stretch, further confirming the robustness observed in Fig. 17.

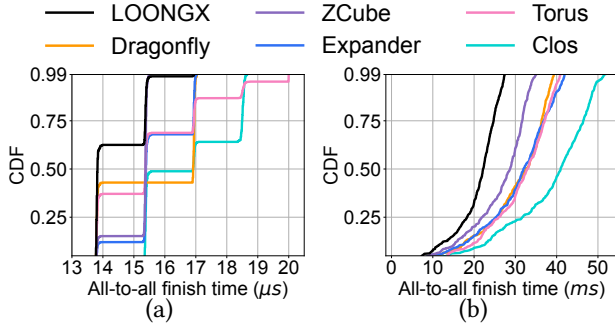


Figure 18: All-to-all traffic finish time for (a) 10 KB and (b) 10 MB flows.

## 6 PROTOTYPE TESTBED

**Testbed setup.** We validate LOONGX on a small-scale testbed and implement other baselines on the same setup to cross-validate simulation results. The testbed consists of an EdgeCore DCS810 switch with Intel Tofino2 and eight servers equipped with Mellanox ConnectX-5 dual-port 100 Gbps NICs. To emulate a 256-nodes topology, we leverage the Tofino2 programmable data plane to virtualize 32–64 logical switches, while using the eight servers to virtualize 256 end hosts. We further use the packet replication engine on Tofino2 to amplify traffic on inter-switch links, which are configured to operate at 100 Gbps. Detailed topology settings are provided in Table 5.

**Workloads.** We synthesize all-to-all workloads based on an MoE architecture with two data parallel groups (DP=2) and eight pipeline parallel groups (PP=8), each containing eight experts (EP=16). All-to-all communication occurs within each PP group, where each expert follows a Top- $k$  routing strategy ( $k = 4$ ) by randomly selecting four target experts from the same group to distribute data. The synthesized traffic includes both short (10 KB) and long (10 MB) flows.

**Performance of short flows.** Fig. 18a shows that under short flows, LOONGX consistently outperforms the baselines. The stair-step pattern reflects the discrete hop counts traversed by each flow. Owing to LOONGX’s dense local connectivity, 98% of the flows complete within one or two hops. Meanwhile, flows in Clos frequently traverse multi stages, resulting in inherently higher hop counts. Torus exhibits the highest 99%-ile all-to-all finish time, primarily due to its larger network diameter.

**Performance of long flows.** Fig. 18b shows smoother curves for long flows, as their performance is dominated by path sharing and available bandwidth rather than packet-level effects in short flows. Therefore, the curves largely track the average hop count of each flow. By combining strong local connectivity with a small number of global shortcuts, LOONGX minimizes hop counts and achieves the best performance. At the 99%-ile all-to-all finish time, LOONGX outperforms ZCube by 8.4%, with 53%, 43%, 49%, and 88% lower

finish times than Expander, Dragonfly, Torus, and Clos, respectively. ZCube benefits from its multi-NIC design, which helps reduce path sharing. Expander delivers moderate performance but is constrained by weak local clustering, while Dragonfly is limited by inter-group bottlenecks despite dense intra-group connectivity. Torus benefits from improved locality under long flows, whereas Clos performs worst due to its longer average path lengths across flows.

## 7 RELATED WORK

**Training-centric network architectures.** Large-scale AI training clusters commonly rely on Clos-like fabrics [16, 34, 43]. To exploit hierarchical bandwidth, rail-optimized and rail-only designs align collectives to preferred communication rails, primarily benefiting structured all-reduce workloads [1, 54]. Google adopts Torus-like topologies in its training supercomputers to match accelerator layouts and provide resiliency [25, 63]. Beyond fixed templates, per-job topology generation has been explored but is challenging to apply in shared multi-tenant environments [55]. More recent training-aware topologies, such as ZCube [60] and Hammingmesh [22], further explore the alignment between AI training communication patterns and network structure.

**Data center networks.** Modern data center networks predominantly use Clos fabrics to provide uniform bandwidth for cloud workloads [2, 3, 18, 19, 33, 42]. To reduce path lengths, Dragonfly-style designs organize routers into groups with high global connectivity to achieve low diameter [26, 49]. Expander-based topologies approximate minimum diameter graphs through irregular connectivity, including Jellyfish and Xpander [5, 21, 50, 52, 58]. In parallel, optical and hybrid architectures support dynamically reconfigurable connectivity, ranging from early electrical-optical designs to microsecond-scale circuit switching and fully optical fabrics [8, 12, 15, 17, 30, 36, 37, 41, 53]. These architectures are primarily designed for cloud workloads with short-lived and randomized flows. Therefore, they fall short when applied to AI training communication, which is long-running, collective-heavy, and highly localized.

## 8 CONCLUSION

In this paper, we revisit the role of randomness in network design for multi-tenant AI training. We show that training communication is largely localized, and that uniformly random expander-style connectivity can over-allocate long-range links at the expense of local bandwidth. LOONGX instead combines structured local neighborhoods with a small number of random shortcuts, achieving low network diameter while preserving strong locality. More broadly, LOONGX offers a new perspective on AI training networks, where structured locality and limited randomness are co-designed rather than treated as competing choices, opening new directions for topology and system co-design in future AI clusters.



## REFERENCES

- [1] [n. d.]. Rail-optimized topology. <https://developer.nvidia.com/blog/doubling-all2all-performance-with-nvidia-collective-communication-library-2-12/>. ([n. d.]).
- [2] Saksham Agarwal et al. 2024. Harmony: A congestion-free datacenter architecture. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 329–343.
- [3] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. 2008. A scalable, commodity data center network architecture. *ACM SIGCOMM computer communication review* 38, 4 (2008), 63–74.
- [4] Mohammad Alizadeh, Albert Greenberg, David A Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. 2010. Data center tcp (dctcp). In *Proceedings of the ACM SIGCOMM 2010 Conference*. 63–74.
- [5] Daniel Amir, Nitika Saran, Tegan Wilson, Robert Kleinberg, Vishal Shrivastav, and Hakim Weatherspoon. 2024. Shale: A practical, scalable oblivious reconfigurable network. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 449–464.
- [6] Serhat Arslan, Yuliang Li, Gautam Kumar, and Nandita Dukkkipati. 2023. Bolt: Sub-RTT Congestion Control for Ultra-Low Latency. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 219–236.
- [7] Chen Avin, Robert Elsässer, Aleksander Figiel, Darya Melnyk, and Stefan Schmid. 2026. Demand-Aware Small-World Networks on Clustered Demands. In *29th International Conference on Principles of Distributed Systems (OPODIS 2025)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 28–1.
- [8] Hitesh Ballani, Paolo Costa, Raphael Behrendt, Daniel Cletheroe, Istvan Haller, Krzysztof Jozwik, Fotini Karinou, Sophie Lange, Kai Shi, et al. 2020. Sirius: A flat datacenter network with nanosecond optical switching. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. 782–797.
- [9] Broadcom. 2023. htsim. <https://github.com/Broadcom/csg-htsim>. (2023).
- [10] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [11] Jiamin Cao et al. 2024. Crux: Gpu-efficient communication scheduling for deep learning training. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 1–15.
- [12] Kai Chen, Ankit Singla, Atul Singh, Kishore Ramachandran, Lei Xu, Yueping Zhang, Xitao Wen, and Yan Chen. 2013. OSA: An optical switching architecture for data center networks with unprecedented flexibility. *IEEE/ACM Transactions on Networking* 22, 2 (2013), 498–511.
- [13] Inho Cho, Keon Jang, and Dongsu Han. 2017. Credit-scheduled delay-bounded congestion control for datacenters. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. 239–252.
- [14] Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, et al. 2022. Glam: Efficient scaling of language models with mixture-of-experts. In *International conference on machine learning*. PMLR, 5547–5569.
- [15] Nathan Farrington, George Porter, Sivasankar Radhakrishnan, Hamid Hajabdolali Bazzaz, Vikram Subramanya, Yeshaiah Fainman, George Papen, and Amin Vahdat. 2010. Helios: a hybrid electrical/optical switch architecture for modular data centers. In *Proceedings of the ACM SIGCOMM 2010 Conference*. 339–350.
- [16] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, et al. 2024. Rdma over ethernet for distributed training at meta scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 57–70.
- [17] Monia Ghobadi, Ratul Mahajan, Amar Phanishayee, Nikhil Devanur, Janardhan Kulkarni, Gireeja Ranade, Pierre-Alexandre Blanche, Houman Rastegarfar, Madeleine Glick, and Daniel Kilper. 2016. Projector: Agile reconfigurable data center interconnect. In *Proceedings of the 2016 ACM SIGCOMM Conference*. 216–229.
- [18] Albert Greenberg, James R Hamilton, Navendu Jain, Srikanth Kandula, Changhoon Kim, Parantap Lahiri, David A Maltz, Parveen Patel, and Sudipta Sengupta. 2009. VL2: A scalable and flexible data center network. In *Proceedings of the ACM SIGCOMM 2009 conference on Data communication*. 51–62.
- [19] Chuanxiong Guo, Guohan Lu, Dan Li, Haitao Wu, Xuan Zhang, et al. 2009. BCube: a high performance, server-centric network architecture for modular data centers. In *Proceedings of the ACM SIGCOMM 2009 conference on Data communication*. 63–74.
- [20] Mark Handley, Costin Raiciu, Alexandru Agache, Andrei Voinescu, Andrew W Moore, Gianni Antichi, and Marcin Wójcik. 2017. Re-architecting datacenter networks and stacks for low latency and high performance. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. 29–42.
- [21] Vipul Harsh, Sangeetha Abdu Jyothi, Inderdeep Singh, and P Godfrey. 2018. Expander datacenters: From theory to practice. *arXiv preprint arXiv:1811.00212* (2018).
- [22] Torsten Hoeftler, Tommaso Bonato, Daniele De Sensi, Salvatore Di Girolamo, Shigang Li, Marco Heddes, Deepak Goel, Miguel Castro, and Steve Scott. 2024. Hammingmesh: A network topology for large-scale deep learning. *Commun. ACM* 67, 12 (2024), 97–105.
- [23] Christian Hopps. 2000. *Analysis of an equal-cost multi-path algorithm*. Technical Report.
- [24] Yanping Huang, Youlong Cheng, Ankur Bapna, et al. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems* 32 (2019).
- [25] Norm Jouppi et al. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th annual international symposium on computer architecture*. 1–14.
- [26] John Kim, William J Dally, Steve Scott, and Dennis Abts. 2008. Technology-driven, highly-scalable dragonfly topology. *ACM SIGARCH Computer Architecture News* 36, 3 (2008), 77–88.
- [27] Jialong Li, Haotian Gong, Federico De Marchi, Aoyu Gong, Yiming Lei, Wei Bai, and Yiting Xia. 2024. Uniform-cost multi-path routing for reconfigurable data center networks. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 433–448.
- [28] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. 2020. Pytorch distributed: Experiences on accelerating data parallel training. *arXiv preprint arXiv:2006.15704* (2020).
- [29] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, et al. 2019. HPCC: High precision congestion control. In *Proceedings of the ACM Special Interest Group on Data Communication*. 44–58.
- [30] Cong Liang, Xiangli Song, Jing Cheng, Mowei Wang, Yashe Liu, Zhenhua Liu, Shizhen Zhao, and Yong Cui. 2024. NegotiaToR: Towards A Simple Yet Effective On-demand Reconfigurable Datacenter Network. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 415–432.
- [31] Xudong Liao, Yijun Sun, Han Tian, et al. 2025. Mixnet: A runtime reconfigurable optical-electrical fabric for distributed mixture-of-experts training. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 554–574.

- [32] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [33] Yunpeng James Liu, Peter Xiang Gao, Bernard Wong, and Srinivasan Keshav. 2014. Quartz: a new design element for low-latency DCNs. *ACM SIGCOMM Computer Communication Review* 44, 4 (2014), 283–294.
- [34] Jie Lu, Jiaqi Gao, Fei Feng, Zhiqiang He, Menglei Zheng, Kun Liu, Jun He, Binbin Liao, Suwei Xu, Ke Sun, et al. 2025. Alibaba stellar: A new generation rdma network for cloud ai. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 453–466.
- [35] Sarah McClure, Sylvia Ratnasamy, and Scott Shenker. 2025. Load Balancing for AI Training Workloads. *arXiv preprint arXiv:2507.21372* (2025).
- [36] William M Mellette, Rajdeep Das, Yibo Guo, Rob McGuinness, Alex C Snoeren, and George Porter. 2020. Expanding across time to deliver bandwidth efficiency and low latency. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. 1–18.
- [37] William M Mellette, Rob McGuinness, Arjun Roy, Alex Forencich, George Papen, et al. 2017. Rotornet: A scalable, low-complexity, optical datacenter network. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. 267–280.
- [38] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, et al. 2019. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM symposium on operating systems principles*. 1–15.
- [39] Van Nguyen and Chip Martel. 2005. Analyzing and characterizing small-world graphs. In *Proceedings of the sixteenth annual ACM-SIAM symposium on Discrete algorithms*. 311–320.
- [40] Jonathan Perry, Amy Ousterhout, Hari Balakrishnan, et al. 2014. Fastpass: A centralized "zero-queue" datacenter network. In *Proceedings of the 2014 ACM conference on SIGCOMM*. 307–318.
- [41] George Porter, Richard Strong, Nathan Farrington, et al. 2013. Integrating microsecond circuit switching into the data center. *ACM SIGCOMM Computer Communication Review* 43, 4 (2013), 447–458.
- [42] Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq, Rui Wang, Jianan Zhang, Virginia Beauregard, Patrick Conner, et al. 2022. Jupiter evolving: transforming google's datacenter network via optical circuit switches and software-defined networking. In *Proceedings of the ACM SIGCOMM 2022 Conference*. 66–85.
- [43] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, et al. 2024. Alibaba hpn: A data center network for large language model training. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 691–706.
- [44] Sudarsanan Rajasekaran, Manya Ghobadi, and Aditya Akella. 2024. {CASSINI}:{Network-Aware} job scheduling in machine learning clusters. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1403–1420.
- [45] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, et al. 2022. DeepSpeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International conference on machine learning*. PMLR, 18332–18346.
- [46] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.
- [47] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, George Porter, and Alex C Snoeren. 2015. Inside the social network's (datacenter) network. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. 123–137.
- [48] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [49] Alexander Shpiner, Zachy Haramaty, Saar Eliad, Vladimir Zdornov, Barak Gafni, and Eitan Zahavi. 2017. Dragonfly+: Low cost topology for scaling datacenters. In *2017 IEEE 3rd International Workshop on High-Performance Interconnection Networks in the Exascale and Big-Data Era (HiPINEB)*. IEEE, 1–8.
- [50] Ankit Singla, Chi-Yao Hong, Lucian Popa, and P Brighten Godfrey. 2012. Jellyfish: Networking data centers randomly. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*. 225–238.
- [51] Frank W Takes and Walter A Kesters. 2011. Determining the diameter of small world networks. In *Proceedings of the 20th ACM international conference on Information and knowledge management*. 1191–1196.
- [52] Asaf Valadarsky, Gal Shahaf, Michael Dinitz, and Michael Schapira. 2016. Xpander: Towards optimal-performance datacenters. In *Proceedings of the 12th International Conference on emerging Networking EXperiments and Technologies*. 205–219.
- [53] Guohui Wang, David G Andersen, Michael Kaminsky, Konstantina Papagiannaki, TS Eugene Ng, Michael Kozuch, and Michael Ryan. 2010. c-Through: Part-time optics in data centers. In *Proceedings of the ACM SIGCOMM 2010 Conference*. 327–338.
- [54] Weiyang Wang, Manya Ghobadi, Kayvon Shakeri, et al. 2023. How to build low-cost networks for large language models (without sacrificing performance)? *arXiv e-prints* (2023), arXiv–2307.
- [55] Weiyang Wang, Moein Khazraee, Zhizhen Zhong, Zhijiao Jia, Dheevatsa Mudigere, Ying Zhang, Anthony Kewitsch, and Manya Ghobadi. 2022. TopoOpt: Optimizing the Network Topology for Distributed DNN Training. *arXiv preprint arXiv:2202.00433* (2022).
- [56] Xizheng Wang, Qingxu Li, Yichi Xu, Gang Lu, Dan Li, Li Chen, Heyang Zhou, et al. 2025. {SimAI}: Unifying Architecture Design and Performance Tuning for {Large-Scale} Large Language Model Training with Scalability and Precision. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. 541–558.
- [57] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. 2022. {MLaaS} in the wild: Workload analysis and scheduling in {Large-Scale} heterogeneous {GPU} clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. 945–960.
- [58] Yiting Xia, Xiaoye Steven Sun, Simbarashe Dzinamarira, et al. 2017. A tale of two topologies: Exploring convertible data center network architectures with flat-tree. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. 295–308.
- [59] Yiting Xia, Xiaoye Steven Sun, Simbarashe Dzinamarira, Dingming Wu, Xin Sunny Huang, and TS Eugene Ng. 2017. A tale of two topologies: Exploring convertible data center network architectures with flat-tree. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. 295–308.
- [60] Zihan Yan, Dan Li, Li Chen, et al. 2025. From ATOP to ZCube: Automated Topology Optimization Pipeline and A Highly Cost-Effective Network Topology for Large Model Training. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 861–881.
- [61] An Yang, Anfeng Li, Baosong Yang, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [62] Qiao Zhang, Vincent Liu, Hongyi Zeng, and Arvind Krishnamurthy. 2017. High-resolution measurement of data center microbursts. In *Proceedings of the 2017 Internet Measurement Conference*. 78–85.
- [63] Yazhou Zu et al. 2024. Resiliency at Scale: Managing Google's TPUv4 Machine Learning Supercomputer. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 761–774.

Table 2: Model configurations.

| Config              | DeepSeek-671B                              | Qwen3-MoE | GLaM-64B/64E | GPT-MoE | GPT-175B |
|---------------------|--------------------------------------------|-----------|--------------|---------|----------|
| <i>Layers</i>       | 61                                         | 94        | 64           | 96      | 96       |
| <i>FFN_H</i>        | 2048                                       | 1536      | 32768        | 2048    | 49152    |
| <i>L</i>            | 4096                                       | 4096      | 1024         | 2048    | 2048     |
| <i>H</i>            | 18432                                      | 4096      | 8192         | 12288   | 12288    |
| <i>Heads</i>        | 128                                        | 64        | 128          | 96      | 96       |
| <i>TP</i>           | 1                                          | 1         | 1            | 1       | 8        |
| <i>PP</i>           | 8                                          | 8         | 8            | 8       | 8        |
| <i>EP</i>           | 64                                         | 32        | 32           | 32      | –        |
| <i>DP</i>           | $World\_size / (TP \times PP \times EP)$   |           |              |         |          |
| <i>Experts</i>      | 256                                        | 128       | 64           | 64      | –        |
| <i>Micro_batch</i>  | 1                                          | 1         | 1            | 1       | 1        |
| <i>Global_batch</i> | $DP \times Micro\_batch \times Batch\_num$ |           |              |         |          |

## APPENDIX

### A Model Configurations

Table 2 summarizes the configurations of the large language models used in our evaluation in Simai’s AICB. *Layers* denotes the number of Transformer blocks. *H* is the hidden dimension, and *Heads* is the number of attention heads. *TP*, *PP*, *EP*, and *DP* denote the sizes of tensor parallelism, pipeline parallelism, expert parallelism, and data parallelism. *FFN\_H* represents the hidden size of the feed-forward network (per expert for MoE models). *L* is the maximum sequence length. *Experts* specifies the number of experts per MoE layer. We additionally report *Micro\_batch*, which is the per-GPU batch size, and *Global\_batch*, computed as  $DP \times Micro\_batch \times Batch\_num$  (set to 8 in our configuration), reflects the total effective batch size with gradient accumulation.

To generate realistic AI training traffic, we align *World\_size*<sup>4</sup> with the simulated cluster topology and configure the parallelism degrees to saturate available resources. The resulting traces explicitly capture communication from DP gradient synchronization, PP stage communication across forward and backward passes, EP token routing and expert aggregation, and TP sharded computation. We also account for TP traffic explicitly: although most TP communication occurs within a node over high-bandwidth links, cross-node tensor sharding can arise when model memory requirements exceed per-node capacity, adding additional load to the network that is reflected in our traces.

### B Enhance htsim for AI Workloads

While the native htsim excels as a high-fidelity packet-level simulator, it is fundamentally limited to modeling flow-level dynamics. It lacks intrinsic support for capturing the complex task-level dependencies that arise in distributed AI training,

<sup>4</sup>For the topologies with 1k, 2.5k, 5k, and 10k-nodes, the total world sizes of all jobs are configured as 1024, 2048, 4096, and 8192, respectively. We ensure a node utilization rate of over 80% across the entire cluster.

Table 3: Full settings for all transport protocols.

| Protocol        | Parameters                                                                                     |
|-----------------|------------------------------------------------------------------------------------------------|
| Common settings | 100 Gbps link speed, 1.5 KB MTU<br>40 cwnd, 500 pkt max queue size                             |
| NDP             | 80 pkt data/credit queue size                                                                  |
| Bolt            | 1 MTU <i>CCThresh</i> , 1 MTU max <i>sm_token</i>                                              |
| HPCC            | $\eta = 95\%$ , <i>max_stage</i> = 5                                                           |
| ExpressPass     | 60 pkt credit queue size<br>40 pkt Aeolus threshold<br>1 <i>winit</i> , 0.1 <i>target_loss</i> |

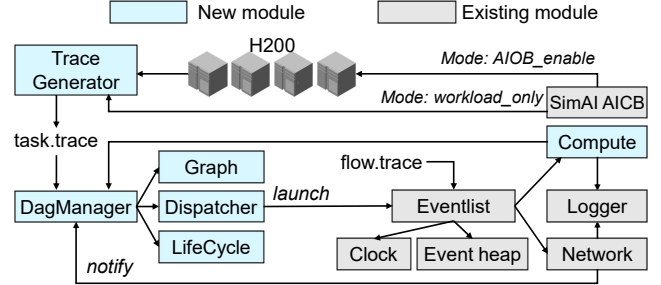


Figure 19: Extend htsim with a trace-driven DAG scheduler.

where communication events can only be triggered after the completion of preceding computational kernels. To bridge the gap between pure network simulation and application-level AI dynamics, we extend htsim with a trace-driven DAG scheduler.

To obtain task-level dependencies, we use SimAI to generate dependency-aware traces. SimAI is configured with the model parameters in Table 2 and produces profiling data in two modes. The *workload\_only* mode captures communication semantics, including traffic volume, collective types, execution order, and group sizes, without executing kernels. The *AI0B\_enable* mode samples realistic layer-wise computation latencies using PyTorch and CUDA on a physical GPU cluster. These outputs are fused by the Trace Generator into a *task.trace* that explicitly encodes computation-communication dependencies.

To process these traces and enforce dependencies, we extend htsim with a task-aware execution layer. The original discrete-event engine, including the Eventlist, Clock, Event heap, and the high-fidelity Network model, is preserved. A new DagManager parses the *task.trace* and dispatches tasks to the Network stack for communication or the virtual Compute module for computation.

Within the DagManager, the Graph module manages DAG dependencies, the Dispatcher module injects events into the Eventlist, and the LifeCycle handles task state transitions. The Compute module estimates kernel completion times based on the Clock. Upon completion of a compute event or network flow, a *notify* signal updates dependency states and releases successor tasks, ensuring strict causal ordering between computation and communication.

**Algorithm 2** BUILDSTRUCTUREDBACKBONE**Require:**

$V$ : set of  $n$  nodes;  
 $k$ : target degree per node ( $k = d - r$ );  
 $\mathcal{L}$ : structured layout policy ( $w$  dimensions).

**Ensure:**

$E_{\text{local}}$ : a  $k$ -regular undirected edge set.

```

1: $E_{\text{local}} \leftarrow \emptyset$, $\deg(u) \leftarrow 0, \forall u \in V$
 // Perform uniform sampling on a w -dimensional unit hypersphere to
 // obtain node embeddings P_u , where $P = \frac{X}{\|X\|_2}, X \sim \mathcal{N}(0, I_w)$
2: $P \leftarrow \text{Place}(V, \mathcal{L})$
3: Compute similarity matrix $S \in \mathbb{R}^{n \times n}$ where $S_{uv} = \langle P(u), P(v) \rangle$
 // Stage A: Local greedy filling via Top- T candidates
4: $\pi(u) \leftarrow$ Top- T neighbors of u sorted by $S_{u,\cdot}$ in descending
 // order, $\forall u \in V$
5: while $\exists u \in V$ s.t. $\deg(u) < k$ do
6: Select $u \in \{x \in V \mid \deg(x) < k\}$
7: Find the first $v \in \pi(u)$ s.t. $\deg(v) < k$ and $(u, v) \notin E_{\text{local}}$
8: if such v exists then
9: $E_{\text{local}} \leftarrow E_{\text{local}} \cup \{(u, v)\}$
10: $\deg(u) \leftarrow \deg(u) + 1$, $\deg(v) \leftarrow \deg(v) + 1$
11: else
12: break
 // Stage B: Global augmentation via masked similarity
13: if $\exists u \in V$ s.t. $\deg(u) < k$ then
14: Initialize mask $M \in \{0, 1\}^{n \times n}$ where $M_{uv} = 1 \iff u \neq v$
15: $M_{uv} \leftarrow 0, \forall (u, v) \in E_{\text{local}}$
16: $M_{u,\cdot} \leftarrow 0, M_{\cdot,u} \leftarrow 0, \forall u \in \{x \mid \deg(x) \geq k\}$
17: while $\exists u \in V$ s.t. $\deg(u) < k$ do
18: Select $u \in \{x \in V \mid \deg(x) < k\}$
19: $C \leftarrow \{x \in V \mid M_{ux} = 1\}$
20: if $C \neq \emptyset$ then
21: $v \leftarrow \arg \max_{x \in C} S_{ux}$
22: else //Random fall-back within unsatisfied nodes
23: $v \leftarrow$ any $x \in \{y \in V \setminus \{u\} \mid \deg(y) < k\}$
24: $E_{\text{local}} \leftarrow E_{\text{local}} \cup \{(u, v)\}$
25: $\deg(u) \leftarrow \deg(u) + 1$, $\deg(v) \leftarrow \deg(v) + 1$
26: $M_{uv} \leftarrow 0, M_{vu} \leftarrow 0$
27: for i in $\{u, v\}$ do
28: if $\deg(i) \geq k$ then $M_{i,\cdot} \leftarrow 0, M_{\cdot,i} \leftarrow 0$
29: return E_{local}

```

**C LOONGX's Construction Details**

The construction process is orchestrated by Alg. 1, which operates in two distinct phases: it first invokes BUILDSTRUCTUREDBACKBONE (Alg. 2) to establish a deterministic, locality-preserving base graph using the structured degree budget  $d - r$  (line 2), and subsequently calls ADDRAMASHORTCUTS (Alg. 3) to superimpose random edges with degree  $r$  for global reachability (line 3). The final topology is produced by merging these two edge sets (line 4) to yield a unified  $d$ -regular graph (line 5).

The Alg. 2 constructs the edge set  $E_{\text{local}}$ . The procedure embeds nodes onto a  $w$ -dimensional unit hypersphere (line 2) and derives a pairwise similarity matrix  $S$  (line 3). The

**Algorithm 3** ADDRAMASHORTCUTS**Require:**

$V$ : set of  $n$  nodes;  
 $r$ : target shortcut degree per node;  
 $E_{\text{avoid}}$ : edge set that shortcuts must not overlap.

**Ensure:**

$E_{\text{short}}$ : an  $r$ -regular Ramanujan expander edge set.

```

1: if $r = 0$ then
2: return $\emptyset$
3: $\mathcal{A} \leftarrow \{(u, v) \mid u, v \in V, u < v, (u, v) \notin E_{\text{avoid}}\}$
 // Candidate pool, try T_{max} times
4: for $t = 1 \dots T_{\text{max}}$ do
5: $E_{\text{short}} \leftarrow \emptyset$, $\deg(v) \leftarrow 0, \forall v \in V$
6: $\mathcal{A}_{\text{shuffled}} \leftarrow \text{Shuffle}(\mathcal{A})$
 // Greedy construction of an r -regular graph
7: for $(u, v) \in \mathcal{A}_{\text{shuffled}}$ do
8: if $\deg(u) < r$ and $\deg(v) < r$ then
9: $E_{\text{short}} \leftarrow E_{\text{short}} \cup \{(u, v)\}$
10: $\deg(u) \leftarrow \deg(u) + 1$, $\deg(v) \leftarrow \deg(v) + 1$
11: if $\forall x \in V, \deg(x) = r$ then
12: break
 // Check r -regularity before spectral test
13: if $\forall x \in V, \deg(x) = r$ then
14: $A \leftarrow$ Adjacency matrix of (V, E_{short})
15: $\lambda_2 \leftarrow \max\{|\lambda| : \lambda \in \text{spec}(A), \lambda \neq r\}$
16: if $\lambda_2 \leq 2\sqrt{r-1}$ then
17: return E_{short}
18: return $\emptyset$

```

construction then proceeds in two stages. Stage A (lines 4–13) prioritizes local density via a greedy filling strategy: nodes identify their Top- $T$  nearest neighbors (line 4), and the algorithm iteratively connects unsatisfied nodes to their best available candidates, exiting early if local options are exhausted (line 12). Stage B (lines 15–34) acts as a global augmentation phase to enforce strict  $k$ -regularity. It initializes a mask to prevent duplicate connections (lines 16–18) and enters a loop to satisfy remaining degree constraints (line 19). For each node, it attempts to connect to the most similar valid peer (line 21), leveraging random selection only as a fallback (lines 24–26), while dynamically updating the mask to maintain graph simplicity (lines 26–29).

The Alg. 3 generates the random shortcut layer. The process begins by populating a pool  $\mathcal{A}$  with all potential edges that do not overlap with the existing structured backbone (line 2). To guarantee optimal spectral expansion, the construction is wrapped in an iterative search loop (line 3). In each iteration, the edge pool is shuffled, and edges are added greedily to build an  $r$ -regular subgraph (lines 5–11). Crucially, once regularity is achieved, the algorithm verifies the graph's expansion properties by computing the second largest eigenvalue  $\lambda_2$  of its adjacency matrix (lines 14–15). The candidate graph  $E_{\text{short}}$  is accepted only if it satisfies or



strictly approaches the Ramanujan bound  $\lambda_2 \leq 2\sqrt{r-1}$  (lines 16–17).

## D Proof of the Clustering Coefficient Decay with Dimension

This section proves that, in LOONGX, the clustering coefficient decreases monotonically as the dimension  $w$  increases.

**THEOREM 3.1 (CLUSTERING COEFFICIENT DECAY WITH DIMENSION).** *For a LOONGX topology with a fixed local degree  $k = d - r$  and a total number of nodes  $N \gg k$ , the local clustering coefficient  $C(w)$  decreases as the number of logical dimensions  $w$  increases.*

**PROOF.** The clustering coefficient  $C(w)$  is formally defined as the probability that two randomly selected neighbors of a node  $u$  are connected. Let  $\mathcal{N}(u)$  denote the neighbor set of  $u$ . For any two neighbors  $v_1, v_2 \in \mathcal{N}(u)$ , the coefficient is given by the conditional probability:

$$C(w) = \Pr(v_1 \sim v_2 \mid v_1, v_2 \in \mathcal{N}(u)) \quad (1)$$

In the geometric construction of LOONGX, where connectivity is determined by proximity within a unit ball  $B(\cdot)$ , this probability is equivalent to the expected intersection volume of the balls  $B(u)$  and  $B(v_1)$ , normalized by the volume of a single ball  $V_d$ .

Let  $d = w - 1$  be the manifold dimension. The distance  $x = \|v_1 - u\|$  follows the probability density function  $f(x) = dx^{d-1}$  for  $x \in [0, 1]$ . For a given distance  $x$ , the normalized intersection volume  $I_d(x)/V_d$  is:

$$\frac{I_d(x)}{V_d} = \frac{2V_{d-1}}{V_d} \int_{x/2}^1 (1-t^2)^{\frac{d-1}{2}} dt \quad (2)$$

The total clustering coefficient is the expectation over  $x$ :

$$C(w) = \int_0^1 dx^{d-1} \left( \frac{2V_{d-1}}{V_d} \int_{x/2}^1 (1-t^2)^{\frac{d-1}{2}} dt \right) dx \quad (3)$$

To demonstrate the monotonic decrease with respect to  $w$ , we observe that for any  $t \in (1/2, 1)$ , the integrand term  $(1-t^2)^{\frac{d-1}{2}}$  decreases exponentially as  $d$  (and thus  $w$ ) increases. Physically, as the dimensionality increases, the volume of the unit ball  $V_d$  concentrates heavily near the boundary, while the overlap region between  $B_u(1)$  and  $B_{v_1}(1)$  occupies a shrinking fraction of the total space.

Asymptotically, the integral is dominated by the behavior at the lower limit  $t = 1/2$ . Therefore:

$$C(w) \lesssim \left( \sqrt{1 - (1/2)^2} \right)^d = \left( \frac{3}{4} \right)^{(w-1)/2} \quad (4)$$

The function  $(3/4)^{(w-1)/2}$  is strictly decreasing. This proves that increasing the logical dimensionality  $w$  of the LOONGX reduces the clustering coefficient  $C(w)$ .  $\square$

---

## Algorithm 4 INCREMENTALEXPANSION

---

### Require:

$LX(n, d, r, w) = (V, E)$ : current graph separating  $E_{\text{base}}$  and  $E_{\text{rand}}$ ;  
 $m$ : number of new nodes to add;  
 $d$ : number of uplinks per node;  
 $r$ : number of random edges per node.

### Ensure:

$LX(n, d, r, w)'$ : updated graph with  $|V| + m$  nodes preserving LoongX properties.

```

1: for $i \leftarrow 1$ to m do
2: Create new node c , $V \leftarrow V \cup \{c\}$
 // Phase 1: Structured Backbone Extension (Local Full-Mesh)
3: Select insertion position pos based on logical layout $\mathcal{L}$
4: Identify local neighbor group $\mathcal{G}$ of size $(d - r)$ around pos
5: $E_{\text{base}} \leftarrow \text{ReMeshToSubgraph}(\mathcal{G} \cup \{c\})$
 // Phase 2: Random Shortcut Extension (Min-Perturbation)
6: if valid matching of size $r/2$ exists in E_{rand} then
7: $\mathcal{M} \leftarrow$ Randomly select $r/2$ disjoint edges from E_{rand}
 // Edges $\{(u_1, v_1), \dots, (u_{r/2}, v_{r/2})\}$ do not share vertices
8: for $(u, v) \in \mathcal{M}$ do
9: $E_{\text{rand}} \leftarrow E_{\text{rand}} \setminus \{(u, v)\}$ // Remove existing edge
10: $E_{\text{rand}} \leftarrow E_{\text{rand}} \cup \{(u, c), (v, c)\}$ // Fold edges to node c
11: $E \leftarrow E_{\text{base}} \cup E_{\text{rand}}$
12: return $LX(n, d, r, w)' = (V, E)$
```

---

## E Incremental Expansion Procedure

To facilitate flexible cluster growth, LoongX implements a low-overhead expansion mechanism that allows new nodes to be integrated online with minimal topological disturbance. The Alg. 4 details the incremental expansion procedure. Iterating through each new node  $c$  (lines 1–2), the process first updates the structured backbone to preserve locality (phase 1): it determines the logical insertion position and identifies the corresponding local neighbor group (lines 3–4), then integrates  $c$  by locally remeshing these nodes into a full-mesh subgraph structure (line 5). Subsequently, to establish global connectivity with minimal network disturbance (phase 2), the algorithm employs an edge-folding strategy: it identifies  $r/2$  disjoint edges from the existing random graph (line 7) and redirects their endpoints to connect with  $c$  (lines 9–10), thereby satisfying the random degree requirement  $r$  while maintaining the topology’s overall expansion properties.

## F Dense Model Performance

We conduct GPT-175B training experiments on 1k nodes with a parallelism configuration of TP=8, PP=8, and DP=16. Fig. 20a illustrates that the iteration time remains remarkably consistent across different topologies. This insensitivity arises because TP communication is highly localized within a single server, thereby decoupling it from the core network, while PP traffic remains minimal. Furthermore, although the network structure affects the efficiency of DP all-reduce, the

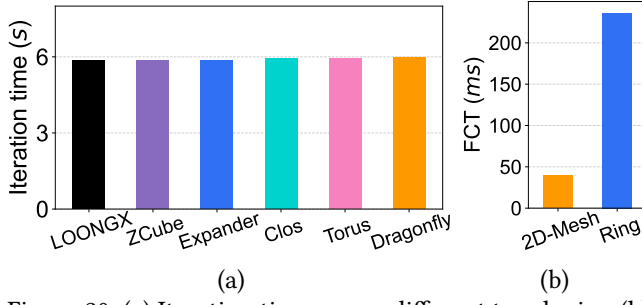


Figure 20: (a) Iteration time across different topologies. (b) FCT of DP all-reduce on LOONGX under the 2D-Mesh and Ring schemes.

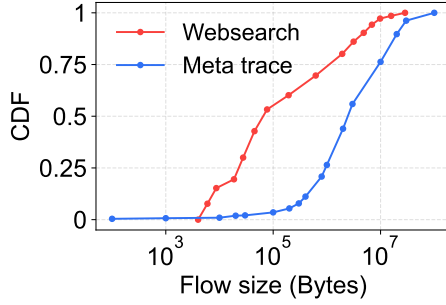


Figure 21: Flow size distributions for Websearch and Meta trace.

pipelining mechanism effectively masks this overhead; consequently, DP traffic constitutes a negligible fraction of the total iteration time for dense models.

Fig. 20b compares the FCT of DP all-reduce on the LOONGX topology under the Ring and 2D-Mesh schemes, where clear performance differences are observed. The results indicate that the 2D-Mesh algorithm dramatically reduces FCT compared to the Ring approach, demonstrating that the structural characteristics of LOONGX are intrinsically well-suited for mesh-based collective communication.

Table 4: Topology configuration and basic property at different scales.

| Topo. Scale (#nodes) | Topology                 | Configuration |                  |         |       |       |                        |              | Property (Node to Node) |      |                     |                        |
|----------------------|--------------------------|---------------|------------------|---------|-------|-------|------------------------|--------------|-------------------------|------|---------------------|------------------------|
|                      |                          | Switch        | Uplink/ Downlink | Radix   | TX/RX | NIC   | Electrical Switch Port | Cost (M. \$) | Diameter                | APL  | APL <sub>fail</sub> | Clustering coefficient |
| 1k                   | LoongX (128,24,6,2)      | 128           | 3:1              | 32      | 5120  | 1024  | 4096                   | 1.948        | 5                       | 3.84 | 3.90                | 0.446                  |
|                      | Expander                 | 128           | 3:1              | 32      | 5120  | 1024  | 4096                   | 1.948        | 5                       | 3.80 | 3.83                | 0.177                  |
|                      | Dragonfly                | 264           | 3:1              | 16      | 5248  | 1024  | 4224                   | 1.984        | 5                       | 4.68 | 4.86                | 0.382                  |
|                      | 6D-Torus                 | 256           | 3:1              | 16      | 5120  | 1024  | 4096                   | 1.948        | 10                      | 6.00 | 6.02                | 0.000                  |
|                      | Clos(Oversubscription=2) | 215           | 1:2              | 15      | 4275  | 1050  | 3225                   | 1.718        | 6                       | 5.85 | 5.85                | –                      |
|                      | Clos(Oversubscription=3) | 164           | 1:3              | 16      | 3680  | 1056  | 2624                   | 1.551        | 6                       | 5.80 | 5.80                | –                      |
|                      | Zcube (Layer=2)          | 64            | 2:1              | 64      | 6144  | 2048  | 4096                   | 2.724        | 3                       | 2.94 | 2.95                | –                      |
|                      | Zcube (Layer=3)          | (200,100)     | 3:1              | (20,30) | 10000 | 3000  | 7000                   | 4.276        | 4                       | 3.72 | 3.76                | –                      |
| 2.5k                 | LoongX (320,24,6,2)      | 320           | 3:1              | 32      | 12800 | 2560  | 10240                  | 4.869        | 5                       | 4.29 | 4.37                | 0.412                  |
|                      | Expander                 | 320           | 3:1              | 32      | 12800 | 2560  | 10240                  | 4.869        | 5                       | 4.06 | 4.13                | 0.071                  |
|                      | Dragonfly                | 510           | 3:1              | 20      | 12750 | 2550  | 10200                  | 4.850        | 5                       | 4.75 | 4.95                | 0.396                  |
|                      | 6D-Torus                 | 648           | 3:1              | 16      | 12960 | 2592  | 10368                  | 4.930        | 9                       | 6.17 | 6.17                | 0.073                  |
|                      | Clos(Oversubscription=2) | 376           | 1:2              | 21      | 10416 | 2520  | 7896                   | 4.168        | 6                       | 5.88 | 5.88                | –                      |
|                      | Clos(Oversubscription=3) | 305           | 1:3              | 20      | 8650  | 2550  | 6100                   | 3.678        | 6                       | 5.87 | 5.87                | –                      |
|                      | Zcube (Layer=2)          | 100           | 2:1              | 100     | 15000 | 5000  | 10000                  | 6.650        | 3                       | 2.96 | 2.97                | –                      |
|                      | Zcube (Layer=3)          | (392,196)     | 3:1              | (28,42) | 27440 | 8232  | 19208                  | 11.733       | 4                       | 3.78 | 3.82                | –                      |
| 5k                   | LoongX (640,24,6,2)      | 640           | 3:1              | 32      | 25600 | 5120  | 20480                  | 9.738        | 6                       | 4.58 | 4.64                | 0.400                  |
|                      | Expander                 | 640           | 3:1              | 32      | 25600 | 5120  | 20480                  | 9.738        | 5                       | 4.35 | 4.43                | 0.035                  |
|                      | Dragonfly                | 876           | 3:1              | 24      | 26280 | 5256  | 21024                  | 9.997        | 5                       | 4.80 | 5.00                | 0.404                  |
|                      | 6D-Torus                 | 1296          | 3:1              | 16      | 25920 | 5184  | 20736                  | 9.860        | 9                       | 6.33 | 6.34                | 0.076                  |
|                      | Clos(Oversubscription=2) | 711           | 1:2              | 27      | 24345 | 5148  | 19197                  | 9.393        | 6                       | 5.90 | 5.90                | –                      |
|                      | Clos(Oversubscription=3) | 504           | 1:3              | 24      | 17280 | 5184  | 12096                  | 7.389        | 6                       | 5.91 | 5.91                | –                      |
|                      | Zcube (Layer=2)          | 144           | 2:1              | 144     | 31104 | 10368 | 20736                  | 13.789       | 3                       | 2.97 | 2.98                | –                      |
|                      | Zcube (Layer=3)          | (648,324)     | 3:1              | (36,54) | 58320 | 17496 | 40824                  | 24.938       | 4                       | 3.83 | 3.87                | –                      |
| 10k                  | LoongX (1280,24,6,2)     | 1280          | 3:1              | 32      | 51200 | 10240 | 40960                  | 19.476       | 6                       | 4.80 | 4.88                | 0.397                  |
|                      | Expander                 | 1280          | 3:1              | 32      | 51200 | 10240 | 40960                  | 19.476       | 6                       | 4.61 | 4.67                | 0.017                  |
|                      | Dragonfly                | 1386          | 3:1              | 28      | 48510 | 9702  | 38808                  | 18.453       | 5                       | 4.83 | 5.05                | 0.411                  |
|                      | 9D-Torus                 | 1728          | 3:1              | 24      | 51840 | 10368 | 41472                  | 19.720       | 11                      | 7.00 | 7.00                | 0.046                  |
|                      | Clos(Oversubscription=2) | 959           | 1:2              | 33      | 41855 | 10208 | 31647                  | 16.789       | 6                       | 5.93 | 5.93                | –                      |
|                      | Clos(Oversubscription=3) | 752           | 1:3              | 32      | 34048 | 9984  | 24064                  | 14.450       | 6                       | 5.92 | 5.92                | –                      |
|                      | Zcube (Layer=2)          | 196           | 2:1              | 196     | 57624 | 19208 | 38416                  | 25.547       | 3                       | 2.98 | 2.99                | –                      |
|                      | Zcube (Layer=3)          | (882,441)     | 3:1              | (42,63) | 92610 | 27783 | 64827                  | 39.600       | 4                       | 3.86 | 3.89                | –                      |

Table 5: Testbed topology configuration.

| Topo. Scale (#nodes) | Topology                 | Switch  | Uplink/Downlink | Radix  | TX/RX | NIC | Electrical Switch Port | Cost (M. \$) |
|----------------------|--------------------------|---------|-----------------|--------|-------|-----|------------------------|--------------|
| 256                  | LoongX (64,6,2,2)        | 64      | 2:1             | 12     | 1024  | 256 | 768                    | 0.411        |
|                      | Expander                 | 64      | 2:1             | 12     | 1024  | 256 | 768                    | 0.411        |
|                      | Dragonfly                | 64      | 2:1             | 12     | 1024  | 256 | 768                    | 0.411        |
|                      | 4D-Torus                 | 64      | 2:1             | 12     | 1024  | 256 | 768                    | 0.411        |
|                      | Clos(Oversubscription=2) | (48,32) | 1:2             | (8,12) | 1024  | 256 | 768                    | 0.411        |
|                      | ZCube (Layer=2)          | 32      | 2:1             | 32     | 1536  | 512 | 1024                   | 0.676        |
|                      |                          |         |                 |        |       |     |                        |              |

Table 6: Cost of network components.

| Link bandwidth | TX/RX (\$) | NIC (\$) | Electrical switch port (\$) |
|----------------|------------|----------|-----------------------------|
| 100 Gbps       | 99         | 650      | 187                         |
| 200 Gbps       | 239        | 1079     | 374                         |
| 400 Gbps       | 659        | 1499     | 1090                        |
| 800 Gbps       | 1399       | 2248     | 1400                        |